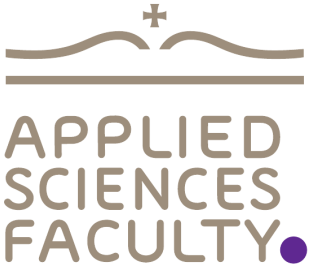
C++ — додаток. Коротко про механізм виклику функцій, на прикладі архітектури x86

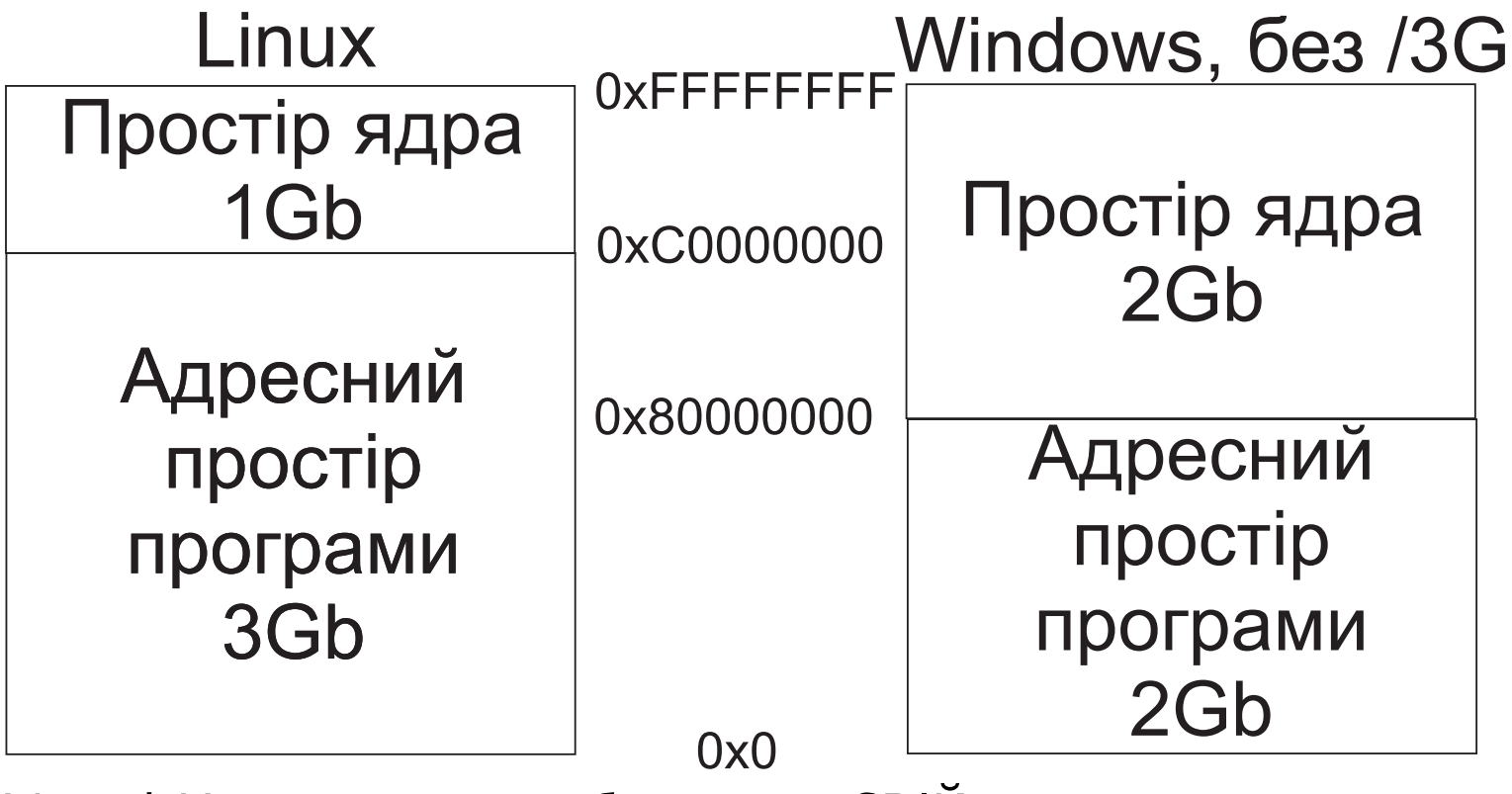




©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

- 32-бітні Linux, Windows та деякі інші сучасні операційні системи надають кожній програмі 4Гб адресного простору.
- Адресні простори різних процесів[1] не перетинаються, знаходяться ніби в паралельних світах, за незначним винятком "shared"-пам'яті, яка використовується для міжпроцесорної взаємодії.
- Зрозуміло, що стільки фізичної пам'яті, щоб кожному процесу виділити 4Гб, немає, та й непотрібно. Реальна, **фізична** пам'ять підставляється ОС в міру потреби. Це якраз так звана віртуальна пам'ять. Важливо, що для програми всі ці процеси підстановки прозорі, тому і ми на них відволікатися не будемо.

[1] Грубо кажучи, процес — це програма, що виконується, разом зі всією інформацією, яка описує її в нутрощах ОС.



Увага! Кожна програма буде мати СВІЙ адресний простір, але системні області відображаються в адресний простір всіх програм.

Детальніше див., наприклад, `http://duartes.org/gustavo/blog/post/anatomy-of-a-program-in-memory`

Linux містить спеціальну утиліту, pmap[2], яка відображає адресний простір вказаного процесу.
Вигляд її виводу наступний:

```
00002b68b377f000      8K rw---    [ anon ]
00002b68b3781000  55140K r----  /usr/lib/locale/locale-archive
00002b68b6d5a000     28K r--s-  /usr/lib64/gconv/gconv-modules.cache
00002b68b6d61000      8K r-x--  /usr/lib64/gconv/ISO8859-15.so
00002b68b6d63000   2044K -----  /usr/lib64/gconv/ISO8859-15.so
00002b68b6f62000      8K rw---  /usr/lib64/gconv/ISO8859-15.so
00002b68b6f64000      8K rw---    [ anon ]
00007fff5273a000     84K rw---    [ stack ]
00007fff527fd000     12K r-x--    [ anon ]
```

- Перша колонка — лінійна адреса початку області[3].
- Друга — розмір області.
- Третя — права доступу до пам'яті. r — дозвіл читати, w — дозвіл писати в цю пам'ять, x — дозвіл виконувати в ній код[4].
- Четверта — якому модулю належить пам'ять, якщо це не частина процесу, тип пам'яті — стек, чи динамічно виділена ([anon])

[2] http://linux.die.net/man/1/pmap

[3] Скріншот зроблено на 64-бітній системі, тому він із 16-х шістнадцяткових цифр. На 32-бітній системі їх би було 8, як на попередньому слайді.

[4] Зверніть увагу — в деякі області програма не має права навіть дивитися, вони використовуються системою.

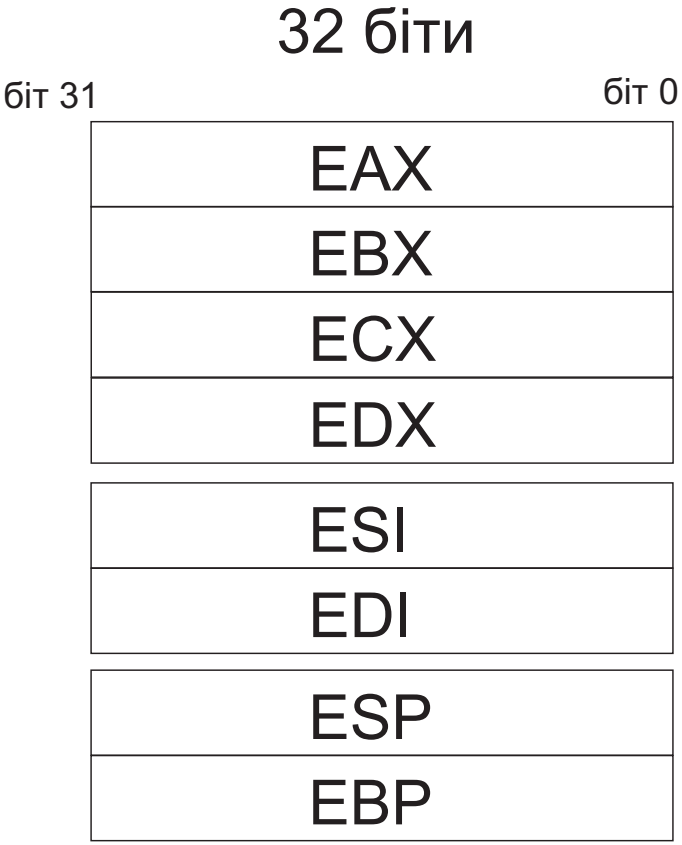
```
[root@west ~]# pmap 19621
19621:   -bash
0000000000400000    712K r-x--  /bin/bash
00000000006b2000     40K rw---  /bin/bash
00000000006bc000     20K rw---    [ anon ]
00000000008bb000     32K rw---  /bin/bash
000000000c5ac000    392K rw---    [ anon ]
0000003561400000    112K r-x--  /lib64/ld-2.5.so
000000356161b000      4K r----  /lib64/ld-2.5.so
000000356161c000      4K rw---  /lib64/ld-2.5.so
0000003561800000   1332K r-x--  /lib64/libc-2.5.so
000000356194d000   2048K -----  /lib64/libc-2.5.so
0000003561b4d000     16K r----  /lib64/libc-2.5.so
0000003561b51000      4K rw---  /lib64/libc-2.5.so
0000003561b52000     20K rw---    [ anon ]
0000003562000000      8K r-x--  /lib64/libdl-2.5.so
0000003562002000   2048K -----  /lib64/libdl-2.5.so
0000003562202000      4K r----  /lib64/libdl-2.5.so
0000003562203000      4K rw---  /lib64/libdl-2.5.so
0000003562400000     12K r-x--  /lib64/libtermcap.so.2.0.8
0000003562403000   2044K -----  /lib64/libtermcap.so.2.0.8
0000003562602000      4K rw---  /lib64/libtermcap.so.2.0.8
0000003857200000     40K r-x--  /lib64/libnss_files-2.5.so
000000385720a000   2044K -----  /lib64/libnss_files-2.5.so
0000003857409000      4K r----  /lib64/libnss_files-2.5.so
000000385740a000      4K rw---  /lib64/libnss_files-2.5.so
00002b68b3739000      8K rw---    [ anon ]
00002b68b377f000      8K rw---    [ anon ]
00002b68b3781000  55140K r----  /usr/lib/locale/locale-archive
00002b68b6d5a000     28K r--s-  /usr/lib64/gconv/gconv-modules.cache
00002b68b6d61000      8K r-x--  /usr/lib64/gconv/ISO8859-15.so
00002b68b6d63000   2044K -----  /usr/lib64/gconv/ISO8859-15.so
00002b68b6f62000      8K rw---  /usr/lib64/gconv/ISO8859-15.so
00002b68b6f64000      8K rw---    [ anon ]
00007fff5273a000     84K rw---    [ stack ]
00007fff527fd000     12K r-x--    [ anon ]
ffffffffff600000   8192K -----    [ anon ]
 total            76492K
```

Поглянемо на пам'ять програми детальніше (на прикладі Linux)

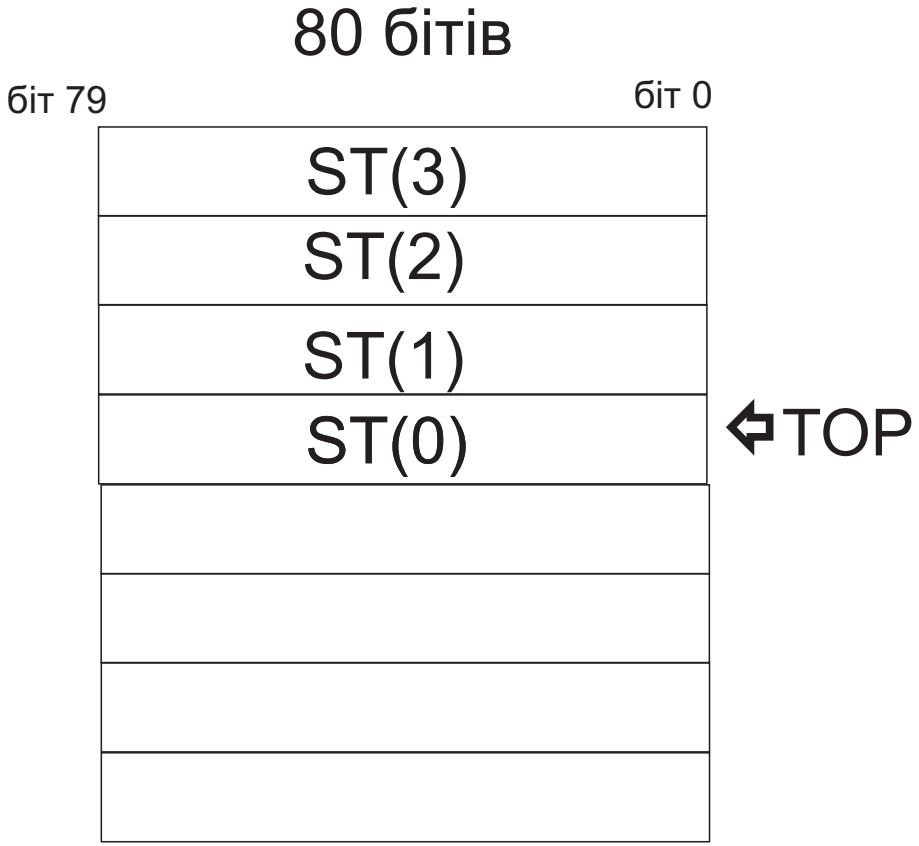


Регістри x86-32

Регістри загального використання



Стек регістрів математичного сопроцесора



Адреса наступної команди, яка буде виконуватися, зберігається в 32-бітному регістрі EIP. Процесор, крім того, має багато інших регістрів — прапорці, EFLAGS, сегментні регістри — CS, SS, DS, ES, FS, GS, регістри керування — CRx, регістри XMM, тощо. Ми їх не розглядатимемо.

Адресація пам'яті

В асемблері x86 звертання до пам'яті може відбуватися так[5]:

$$\begin{Bmatrix} CS: \\ DS: \\ SS: \\ ES: \\ FS: \\ GS: \end{Bmatrix} \left[\begin{Bmatrix} EAX \\ EBX \\ ECX \\ EDX \\ ESP \\ EBP \\ ESI \\ EDI \end{Bmatrix} + \left[\begin{Bmatrix} EAX \\ EBX \\ ECX \\ EDX \\ EBP \\ ESI \\ EDI \end{Bmatrix} * \begin{Bmatrix} 1 \\ 2 \\ 4 \\ 8 \end{Bmatrix} \right] + [\text{displacement}] \right]$$

Приклади:

- `[123456]` — безпосередньо вказана адреса: починаючи з байта номером 123456.
- `[ESI]` — адресу взяти з регістра ESI.
- `[ESI - 4]` — адреса це значення, яке знаходиться в ESI, мінус 4.
- `[EAX + 2*ESI + 33]`
- `[EDX + 15]`

[5] Зображення взято з вікі: `http://en.wikipedia.org/wiki/X86`

Деякі команди — 1

- **add** — додати два значення, **sub** — відняти два значення. Результат кладеться в лівий аргумент.

```
add     EAX, EBX      ; EAX = EAX + EBX
add     EDX, 5        ; EDX = EDX + 5
add     ECX, [EBP+4*ESI]    ; ECX = ECX + вміст памяті'
                            ; за адресою [EBP+4*ESI]
add     [EBP], ESI    ; До памяті', на яку вказує EBP,
                            ; буде додано вміст ESI
```

- **push** — покласти значення або вміст регістру чи пам'яті в стек, **pop** — забрати значення з вершини стеку в регістр або пам'ять.

```
push    EAX
push    [EDI]
push    333h

pop     [ESI]
pop     EBX
```

Деякі команди — 2

- **call** — виклик підпрограми:

```
call    12345678h   ; Виклик за безпосередньою адресою
call    [EBX]       ; Опосередкований виклик,
                    ; адреса підпрограми в EBX
```

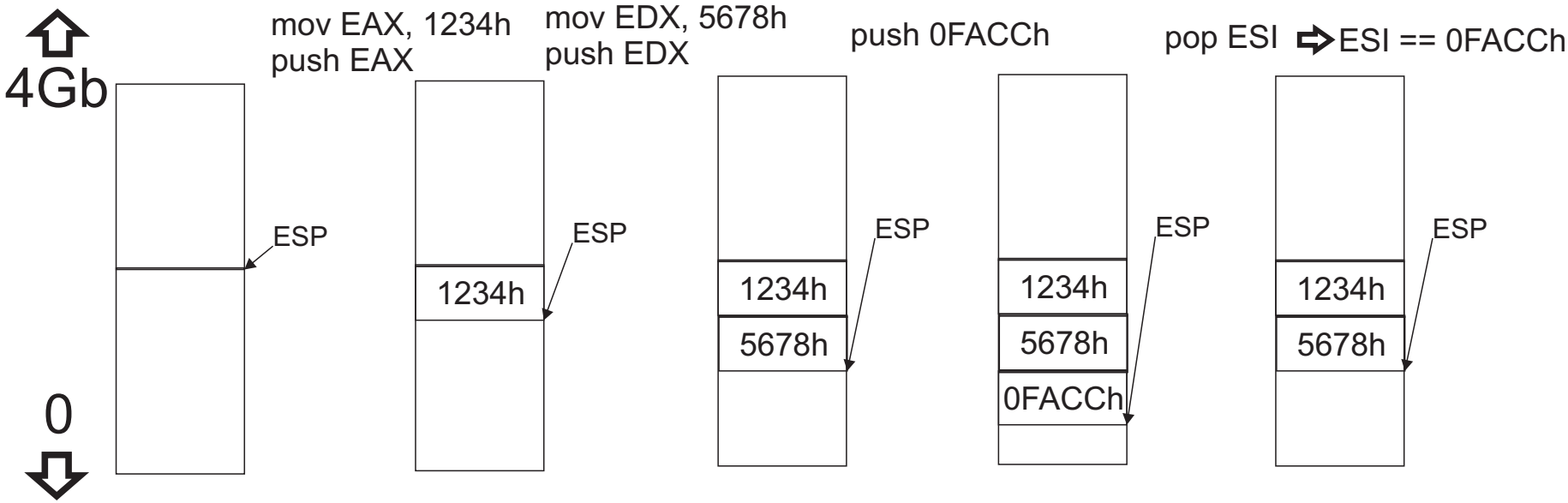
  Під час виклику адреса повернення кладеться на стек.

- **ret** — повернутися із підпрограми. Адреса повернення береться із стеку.

```
ret             ; // Взяти зі стеку останню збережену адресу
ret         4   ; // Після того, як зі стеку дістали адресу
                // повернення, відкинути ще вказану кількість
                // байт
```

Цей список — навіть не початок! Детальніше див. тут: `http://ref.x86asm.net/`, скажімо у варіанті coder32.

- Поточна вершина стеку знаходиться в ESP.
- Коли виконується команда push, від ESP віднімається[6] 4 та за адресою [ESP] зберігається значення, яке push-ається в стек.
- Тобто, стек росте ВНИЗ — від більших адрес до менших.
- pop поводиться зворотним чином — в свій операнд кладе [ESP] і збільшує ESP на 4.
- push ESP кладе на стек значення ESP до операції.



[6] В різних режимах це значення може бути різним, не розглядатимемо інші комбінації.

Виклик функції

Розглянемо наступний код:

```
void fn_l3(){
}

void fn_l2(){
    fn_l3();
}

void fn_l1(){
    fn_l2();
}

int main(){
    fn_l1();

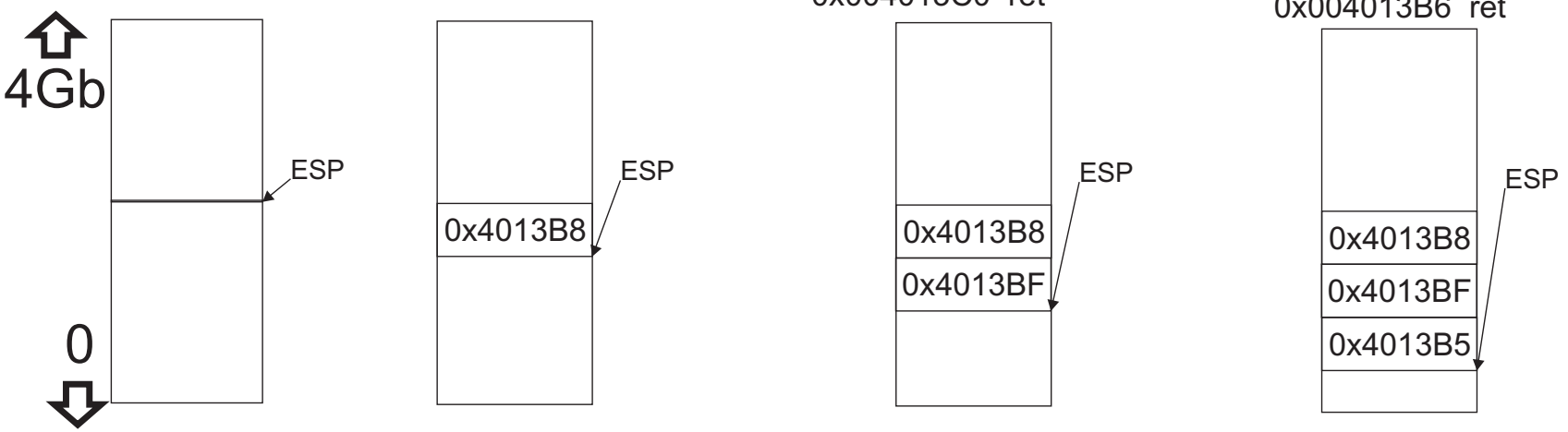
    return 0;
}
```

Виклик функції

0x004013CC: call **0x4013B7**
<fn_l1()>
0x004013B8: <наступна команда>

0x004013B7 push ebp
0x004013B8 mov ebp,esp
0x004013BA call **0x4013AD**
<fn_l2()>
0x004013BF pop ebp
0x004013C0 ret

0x004013AD push ebp
0x004013AE mov ebp,esp
0x004013B0 call **0x4013A8**
<fn_l3()>
0x004013B5 pop ebp
0x004013B6 ret



Calling conventions — угоди про виклики

- Угоди про виклики — домовленість про те, як передаватимуться у функцію параметри, та як повертатиметься результат, як кодуються імена функцій в об'єктних файлах, тощо.
- Їх існує багато, стандартизовані вони лише в загальних рисах. При тому, код, який викликає функцію та сама функція повинні дотримуватися єдиних домовленостей. Зазвичай про це турбується компілятор, однак слід бути уважним, користуючись готовими скомпільованими модулями.
- Декілька прикладів угод про виклики: cdecl, syscall, pascal, stdcall, fastcall (є варіанти від Microsoft та Borland), thiscall, і т.д. і т.п. Коротко можна почитати тут: `http://en.wikipedia.org/wiki/Calling_convention`, `http://en.wikipedia.org/wiki/X86_calling_conventions`,а детальніше тут: `http://www.agner.org/optimize/` — "Calling conventions for different C++ compilers and operating systems" by Agner Fog — детальний, повний та доволі точний довідник.
- Ми розглянемо їх на одному прикладі — cdecl, типової угоди про виклики для C-програм, при чому, лише для архітектури процесорів x86.

cdecl for x86

- Аргументи функціям передаються через стек.
- Цілі числа та вказівники повертаються за допомогою регістра EAX. Числа з рухомою комою повертаються через спеціальний регістр сопроцесора, ST0.
- Функція може вільно змінювати EAX, ECX і EDX, (так звані scratch-регістри, або caller-saved) але зобов'язана зберегти і відновити всі інші регістри, якщо модифікує їх (callee-saved)[7].
- Аргументи передаються справа наліво.
- За очистку стеку від параметрів відповідає код, що викликає функцію (caller)
- Про передачу більш складних типів коротко буде сказано пізніше, але важливо знати, що власне у її правилах різні компілятори поводяться трішки по різному, навіть в рамках cdecl.
- В деяких компіляторах можна змусити використовувати cdecl ось так:

  void _cdecl fn();

  УВАГА! _cdecl — не є частиною стандарту C/C++! Це платформно-залежне розширення, хоча й популярне.

[7] Регістри сопроцесора мають бути порожніми на вході, зайнятим на виході може бути хіба ST0, якщо функція повертає число з рухомою комою.

Розглянемо функцію:

```
void fn(int a, int b, int c);
```

Викличемо її, для простоти, так:

```
fn(1, 2, 3);
```

Передача аргументів їй може відбуватися якось так:

Безпосередньо запхати в стек:

```
push    3
push    2
push    1
call    <fn>
```

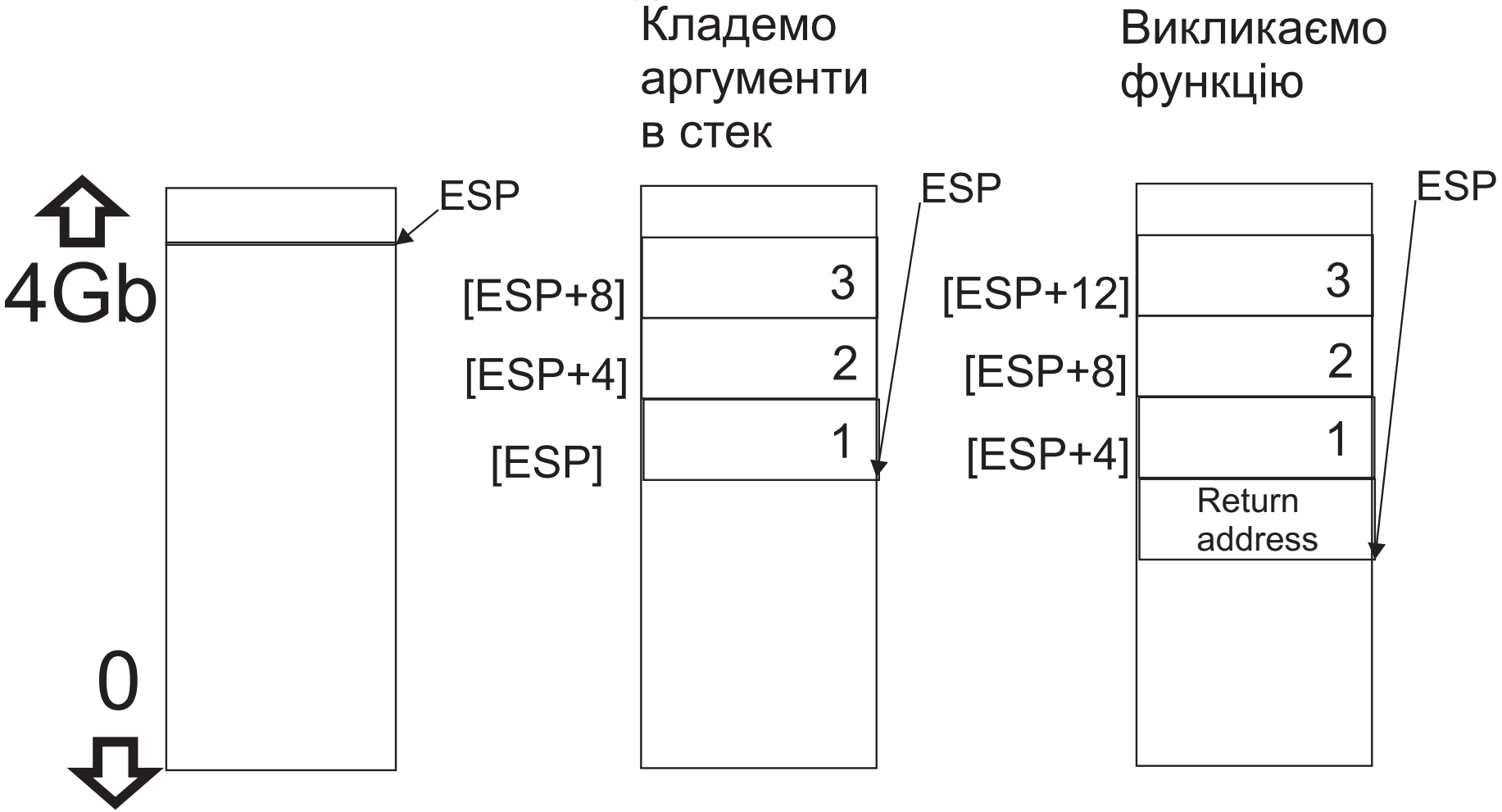
Трішки зекономити:

```
sub     esp,12      ; 4*3
mov     [esp+8],3
mov     [esp+4],2
mov     [esp],1
call    <fn>
```

GCC вимагає, щоб стек був вирівняний на межу 16 байт, тому виділяє зразу 16 = 10h

```
sub     esp,10h     ; 16
mov     [esp+8],3
mov     [esp+4],2
mov     [esp],1
call    <fn>
```

Яким би способом аргументи не були покладені в стек, картина виглядатиме так:



Таким чином, відштовхуючись від значення ESP на початку її виконання, функція завжди може знайти свої аргументи.

Доступ до аргументів

Подивимося на це з точки зору функції:

```
void fn(int a, int b, int c)
{
    a = a + b + c;
}
```

Для простоти обійдемося поки без локальних змінних.

Компілятор може згенерувати код ось такого типу:

```
push   ebp
mov    ebp,esp

mov    eax,DWORD PTR [ebp+0Cx0]
mov    edx,DWORD PTR [ebp+0x08]
add    eax,edx
add    eax,DWORD PTR [ebp+0x10]
mov    DWORD PTR [ebp+0x08],eax

pop    ebp
ret
```

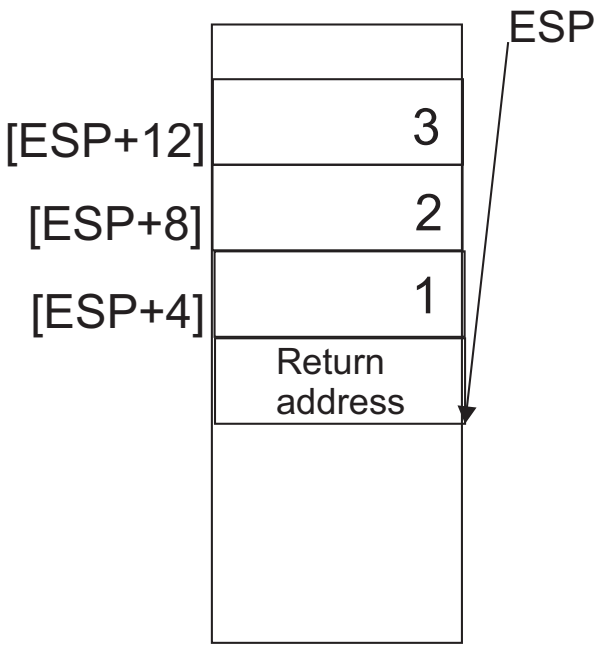
Кадр стеку — stack frame

Розглянемо наступний код:

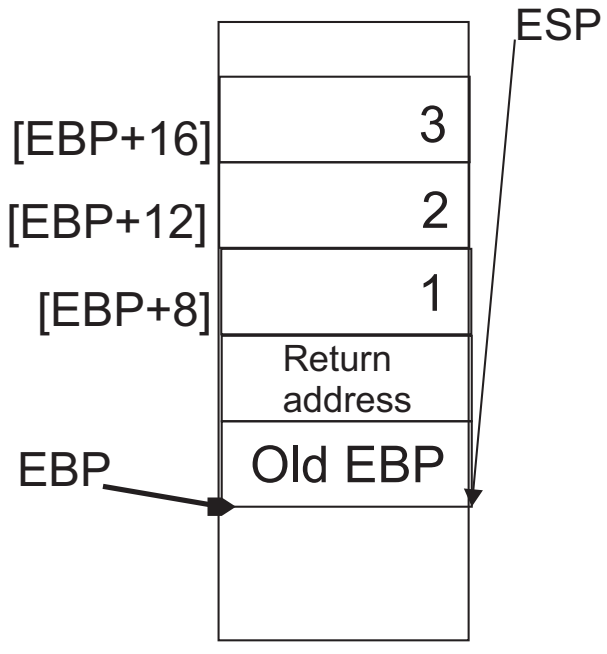
```
push    ebp         ; Зберігаємо EBP
mov     ebp,esp     ; Налаштовуємо кадр стеку
        .....
pop     ebp         ; Відновлюємо EBP
ret
```

- ESP буде змінюватися в процесі роботи функції, тому хоча локальні параметри на початку функції починаються з ESP+4, потім доступатися до них може бути складно.
- Тому варто десь зберегти його значення на початку функції.
- Архітектура x86 рекомендує використовувати для цього регістр EBP.
- Однак, згідно угоди про виклики cdecl, його зберігати та відновлювати повинна сама функція. Тому, вона починає із того, що зберігає EBP в стеку, а завершує, перед виходом, відновлюючи його.

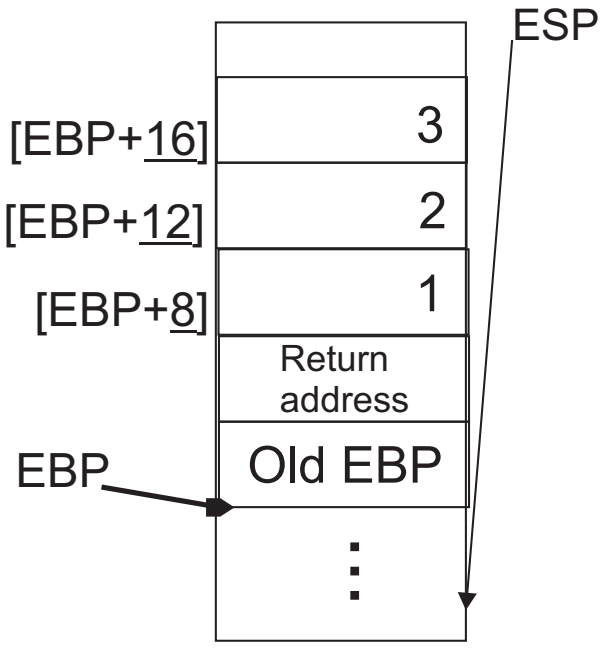
На початку функції



push EBP
mov EBP, ESP



Тіло функції
.....



Зауважте, звертатися до аргументів функції можна не залежно від змін ESP.

Нагадаємо собі реалізацію функції, яку розглядаємо:

```
void fn(int a, int b, int c)
{
    a = a + b + c;
}
```

```
push   ebp
mov    ebp,esp

mov    eax,DWORD PTR [ebp+0Cx0] ; 0x0C = 12, b
mov    edx,DWORD PTR [ebp+0x08] ; 0x08 = 8,  a
add    eax,edx
add    eax,DWORD PTR [ebp+0x10] ; 0x10h = 16, c
mov    DWORD PTR [ebp+0x08],eax ; Зберегти в a

pop    ebp
ret
```

Завершуючи із передачею простих аргументів

```
fn_3args(1,2,3);
```

```
; Готуємо аргументи функції
mov    dword ptr [esp+0x8],0x3          ; Тре-
тій
mov    dword ptr [esp+0x4],0x2          ; Дру-
гий
mov    dword ptr [esp],0x1
ший
call   0x4013ce <fn_3args(int, int, int)>
```

```
int a1 = 5, b1 = 55, c1 = 555;
fn_3args(a1,b1,c1);
```

```
; Ініціалізуємо змінні
mov    dword ptr [esp+0x18],0x5
; esp+0x18 – адреса a1
mov    dword ptr [esp+0x14],0x37
; esp+0x14 – адреса b1
mov    dword ptr [esp+0x10],0x22b
; esp+0x10 – адреса c1
; Готуємо аргументи функції
mov    eax,dword ptr [esp+0x10]
mov    dword ptr [esp+0x8],eax          ; Тре
тій, Пер-
mov    eax,dword ptr [esp+0x14]
mov    dword ptr [esp+0x4],eax          ; Дру
гий
mov    eax,dword ptr [esp+0x18]
mov    dword ptr [esp], eax
ший
call   0x4013ce <fn_3args(int, int, int)>
```

Зрозуміло, що, із ввімкненою оптимізацією, компілятор викине зайві переміщення

Повернення значень простих типів

```
int  fn_3args_ret
        (int a, int b, int c){
    a = a + b + c;
    return a;
}
```

```
push   ebp
mov    ebp,esp
mov    eax,dword ptr [ebp+0xc]
mov    edx,dword ptr [ebp+0x8]
add    eax,edx
add    eax,dword ptr [ebp+0x10]
mov    dword ptr [ebp+0x8],eax
mov    eax,dword ptr [ebp+0x8]
pop    ebp
ret
```

```
int d;
d = fn_3args_ret(1,2,3);
```

```
mov    dword ptr [esp+0x8],0x3
mov    dword ptr [esp+0x4],0x2
mov    dword ptr [esp],0x1
call   0x4013e1
    <fn_3args_ret(int, int, int)>
mov    dword ptr [esp+0x1c],eax
```

Локальні змінні

Код вище вже містить натяки, що локальні змінні теж знаходяться в стеку.

Розглянемо модифікацію нашої функції:

```
int  fn_locals(int a, int b, int c)
{
    int r;
    r = a + b + c;
    return r;
}
```

```
push   ebp
mov    ebp,esp

sub    esp,0x10

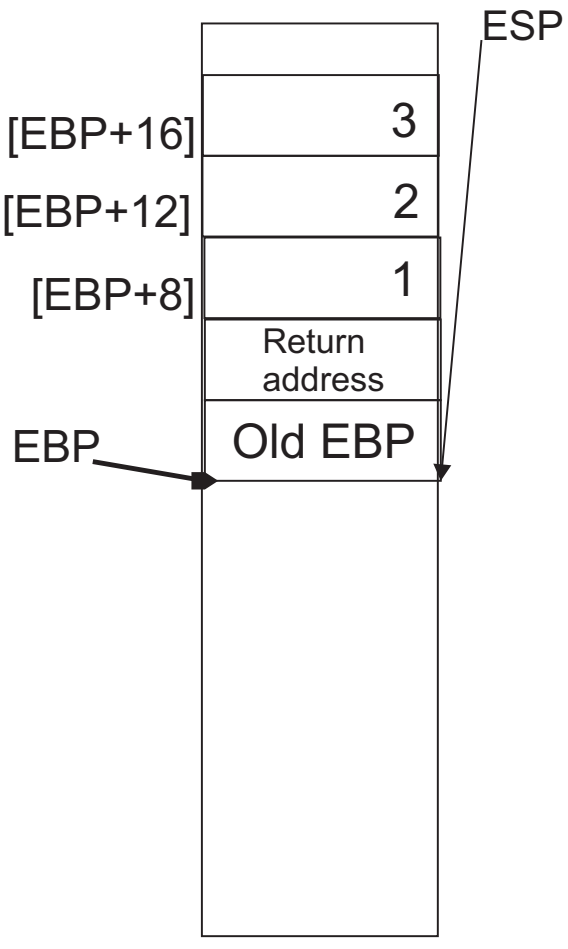
mov    eax,dword ptr [ebp+0xc]
mov    edx,dword ptr [ebp+0x8]
add    eax,edx
add    eax,dword ptr [ebp+0x10]

mov    dword ptr [ebp-0x4],eax

mov    eax,dword ptr [ebp-0x4]

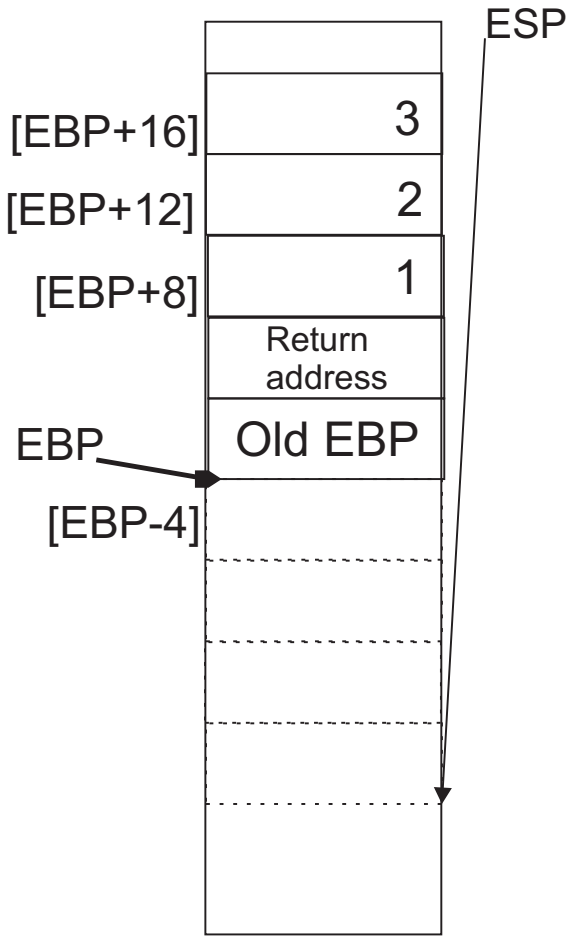
mov        esp, ebp
pop    ebp
ret
```

push EBP
mov EBP, ESP

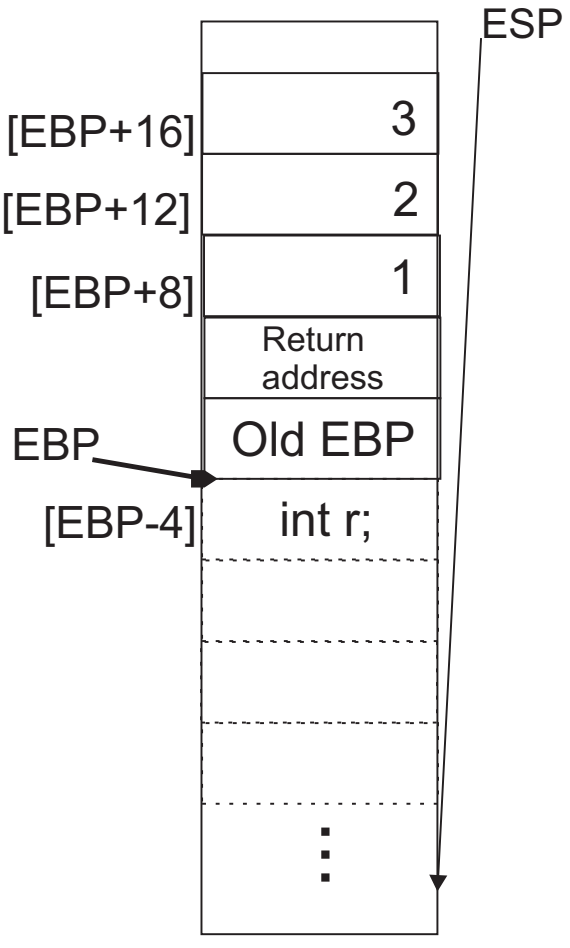


Виділяємо місце під локальні змінні:

sub ESP,0x10



Звертання до локальної змінної
mov eax, [EBP-0x4]



Віднявши від ESP – резервуємо місце в стеку, додавши – звільняємо, відкидаючи вміст.

Готуємо кадр стеку:

Виділяємо пам'ять для локальних змінних:

Зберігаємо результат в локальній змінній r, до якої звертаємося відносно границі кадру стеку:

Кладемо в EAX результат роботи із локальної змінної:

Очищаємо стек від локальних змінних та відновлюємо EBP. Часто цю пару інструкцій замінюють на еквіваленту їй команду **leave**

Завершуємо функцію

```
push   ebp
mov    ebp,esp

sub    esp,0x10

       ; Розрахунок результату
mov    dword ptr [ebp-0x4],eax

mov    eax,dword ptr [ebp-0x4]

mov    esp, ebp
pop    ebp
```

leave

```
ret
```

Коротко про передачу структур та класів

Детально тему не розглядатимемо, якщо цікаво — див. згадану роботу "Calling conventions for different C++ compilers and operating systems", розділ "7.1 Passing and returning objects".

Чисто для ілюстрації:

- 32-бітний VS передає всі структури та класи, просто копіюючи їх в стек.
- 32-бітний GCC копіює структури та класи в стек повністю, крім випадків коли клас має конструктор копіювання, деструктор або віртуальні функції — тоді він створює копію десь в пам'яті та кладе в стек вказівник на цю копію[8].

[8] Нагадаю, що для програміста все це відбувається повністю прозоро.

Додаткова література:

- http://en.wikipedia.org/wiki/Calling_convention
- http://en.wikipedia.org/wiki/X86_calling_conventions
  http://en.wikipedia.org/wiki/Function_prologue
- http://www.agner.org/optimize/ — "Calling conventions for different C++ compilers and operating systems" by Agner Fog, детальний, повний та доволі точний довідник.
- Вікі-книжка про дизасемблювання коду x86: http://en.wikibooks.org/wiki/X86_Disassembly/The_Stack, http://en.wikibooks.org/wiki/X86_Disassembly/Functions_and_Stack_Frame_Examples, http://en.wikibooks.org/wiki/X86_Disassembly/Calling_Conventions, http://en.wikibooks.org/wiki/X86_Disassembly/Calling_Convention_Examples.
- http://wiki.osdev.org/Stack — розмова про стек процесорів сімейства x86 на сайті любителів розробки власних операційних систем.
- http://www.unixwiz.net/techtips/win32-callconv-asm.html і http://www.unixwiz.net/techtips/win32-callconv.html. Увага, використано "діалект" асемблера від AT&T, зокрема, порядок аргументів зворотній до використаного вище.